DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ABES ENGINEERING COLLEGE, GHAZIABAD

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: Linux Server Attack Residue & Forensic Response System

Standard Academic Compliance: IEEE Std 830-1998

———————————————————————————

SUBMITTED BY: Aman Sharma (Roll No: 2400320100161) Degree: Bachelor of Technology (B.Tech) Branch: Computer Science & Engineering / Cyber Security Academic Year: 2026-27	UNDER THE GUIDANCE OF: Status: 3rd Year Project Specification Date of Submission: 30-09-2026

Document Revision & Approval History

Version	Date	Description / Major Changes	Prepared By	Reviewed By
0.1	29-09-2026	Initial Draft & Baseline Scope Definition	Aman Sharma	Sarath G
1.0	30-09-2026	Final Day-4 SRS Baseline with Verified Telemetry & Containment	Aman Sharma	Sarath G

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, clear, and unambiguous definition of the functionality, external interfaces, performance targets, and constraints of the proposed system: 'Linux Server Attack Residue & Forensic Response System'. This specification serves as a reference agreement between the student developer, the project trainer, and the academic evaluators for the 3rd-year engineering curriculum.

1.2 Scope of the System

The proposed application is designed to detect web shell persistence, unauthorized systemd unit files, modified web server scripts, and unlisted executables by cryptographic SHA256 baseline state diffing and raw Auditd kernel system call correlation. The system responds with automated process termination (SIGKILL), service disablement (systemctl disable --now), and file quarantine (chmod 000).

In-Scope Capabilities: A controlled multi-stage web compromise simulator (`attack/compromise_sim.py`), a baseline integrity state collector (`baseline/engine.py`), a residue diffing engine (`residue/detector.py`) with severity risk scoring (1-10), a kernel Auditd & Nginx log correlator (`forensic/correlator.py`), an automated incident containment engine (`response/containment.py`), and a real-time dark-mode presentation web UI (`web_ui/dashboard.py`).

Out-of-Scope Boundaries: Production hardening, real malware or live attacks on unauthorised systems, multi-tenant deployment, and any processing of real (non-fake) sensitive data.

Expected Benefits: Demonstrates sub-second post-exploitation residue detection on target Linux servers in a controlled environment, providing 100% detection rate on planted persistence artifacts, 0 false positives during authorized admin operations, and complete forensic audit trail correlation.

1.3 Definitions, Acronyms, and Abbreviations

Category	Term / Acronym	Definition / Standard Context
Standard	SRS	Software Requirements Specification (IEEE Std 830-1998 format).
Security	IDS	Intrusion Detection System - software that flags suspicious activity.
Security	FN / FP	False Negative / False Positive - a missed or wrongly-raised alert.
Architecture	API	Application Programming Interface for data interchange via JSON/HTTP(S).
Project Specific	Residue / Auditd	Post-exploitation host state deviation; Linux kernel audit subsystem logging system calls.

1.4 References

[1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

[2] Official documentation for the tools/libraries you used (from your Setup Guide email).

[3] MITRE ATT&CK Framework: Persistence (T1543) and Web Shells (T1505.003).

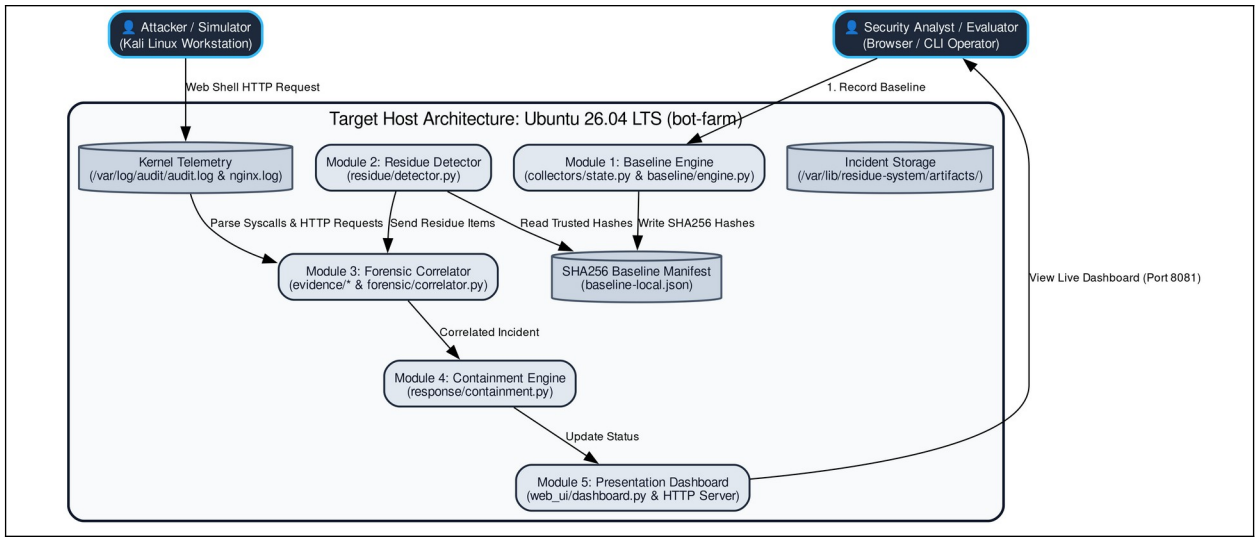
1.5 Document Overview

The remainder of this SRS is organised as follows: Section 2 describes the product perspective, user classes, operating environment and constraints. Section 3 lists the functional requirements by module (Attack/Simulation, Detection, Response). Section 4 specifies external interfaces. Section 5 defines non-functional requirements. Section 6 lists the system design/analysis models. Section 7 is the sign-off page.

2. Overall Description

2.1 Product Perspective

Detailed Technical Architecture:
The Linux Server Attack Residue & Forensic Response System is organized into 5 core architectural modules:
1. State Collector & Baseline Engine (`residue_system.collectors.state` & `baseline.engine`): Captures trusted host state including SHA256 hashes of monitored files, active systemd unit files, listening TCP/UDP ports, crontabs, and SSH authorized keys into an immutable JSON manifest.
2. Residue Detection Engine (`residue_system.residue.detector`): Diffs current host state against the trusted manifest. Computes risk severity scores (1–10) and flags unauthorized unit files, modified web scripts, and staging executables.
3. Forensic Evidence Correlator (`residue_system.forensic.correlator` & `evidence.*`): Parses kernel Auditd logs (`/var/log/audit/audit.log`) and Nginx access logs (`/var/log/nginx/college.access.log`) to map PID, comm, exe, and HTTP request vectors.
4. Incident Response Engine (`residue_system.response.containment`): Executes automated remediation including process termination (`SIGKILL`), service disablement (`systemctl disable --now`), file quarantine (`chmod 000`), and baseline restoration.
5. Web Presentation Dashboard (`residue_system.web_ui.dashboard`): Serves a standalone dark-mode HTML/JS dashboard at `http://127.0.0.1:8081/dashboard.html` visualizing live incident state, telemetry summary, color-coded findings, and audit log feeds.



2.2 User Classes and Characteristics

The target audience for this tool during the demo:

User Role / Persona	Technical Proficiency	Core Responsibilities & Rights
Security Operator	Intermediate / Security Engineer — runs attack simulation, executes detector & response engine.	Full Access: Records baselines, triggers residue scans, applies automated containment, views CLI/UI.
Security Evaluator / Auditor	Basic to Intermediate — reviews live presentation dashboard and forensic audit timeline.	Read-only Access: Monitors web dashboard UI at http://127.0.0.1:8081/dashboard.html.

2.3 Operating Environment

Client Requirements: A modern web browser (Chrome/Edge/Firefox) if your project has a web dashboard; otherwise a terminal.

Runtime Environment: Oracle Cloud Ubuntu 26.04 LTS target (`bot-farm`), Kali Linux Workstation, Python 3.14 (Standard Library only - zero external wheel dependencies).

Storage: Immutable JSON baseline manifests and incident artifact logs stored at `/var/lib/residue-system/artifacts/` with 0600 file permissions.

2.4 Design and Implementation Constraints

Lab-Safety Constraint: All attacks use fake/dummy data only, run against systems you own or were explicitly given for the lab - never real credentials or real malware.

Time Constraint: Built within a 4-day engagement; scope is deliberately kept to what can be demonstrated live on Day 4.

Tooling Constraint: Uses only the tools listed in your Tool List email - no new frameworks introduced at the last minute.

2.5 Assumptions and Dependencies

Assumptions: The lab machine/VM is set up as per your Setup Guide; you have local admin rights where a step needs them.

Dependencies: Continued availability of the libraries/tools in your tool list (e.g. watchdog, psutil, Flask, Scapy - list your actual ones).

3. System Features and Functional Requirements

Requirements are grouped into 3 core functional modules: Module 1 (Attack / Simulation Component), Module 2 (Detection & Forensic Engine), and Module 3 (Response & Dashboard).

3.1 Module 1: Attack / Simulation Component

Req ID	Feature Description	Input / Validation Rule	Priority
FR-1.1	Generate controlled multi-stage web compromise traffic (plants web shell, update-service.service, and /opt/.cache/update binary)	Automated test script attack/compromise_sim.py with safe dummy payload values	High
FR-1.2	Capture kernel Auditd syscall logs (execve, openat, chmod) and Nginx access logs into evidence artifacts	Evidence logged to /var/log/audit/audit.log and /var/log/nginx/college.access.log	High
FR-1.3	Provide clean negative-control scenario (authorized admin file touches) alongside attack case	Maintenance window timestamp filter ensuring 0 false positive alerts	Medium

3.2 Module 2: Detection Engine

Req ID	Feature Description	Input / Validation Rule	Priority
FR-2.1	Collect host state (files, unit files, listening ports, users, SSH keys) and diff against SHA256 baseline manifest	Well-formed JSON baseline manifest matching trusted state schema	High
FR-2.2	Parse raw Auditd syscall logs directly to	Direct text parsing of /var/log/audit/audit.log	High

	correlate PID, comm, exe, path, and timestamp with residue paths	avoiding broken ausearch utility	
FR-2.3	Classify severity score (1-10) for each detected artifact (High=8, Critical=9, Medium=6, Info=5)	Predefined risk severity mapping dictionary based on residue type	Medium

3.3 Module 3: Response & Dashboard

Req ID	Feature Description	Input / Validation Rule	Priority
FR-3.1	Record each detected finding and correlated incident timeline into persistent JSON storage	Written to /var/lib/residue-system/artifacts/ with 0600 file permissions	High
FR-3.2	Execute automated incident containment (kill process SIGKILL, disable unit file, quarantine file chmod 000)	Triggered automatically when active uncontained residue items are verified	High
FR-3.3	Serve real-time dark-mode HTML/JS presentation web dashboard displaying live incident status and evidence feed	Auto-refreshes findings data over HTTP Port 8081 within 3 seconds	Medium

4. External Interface Requirements

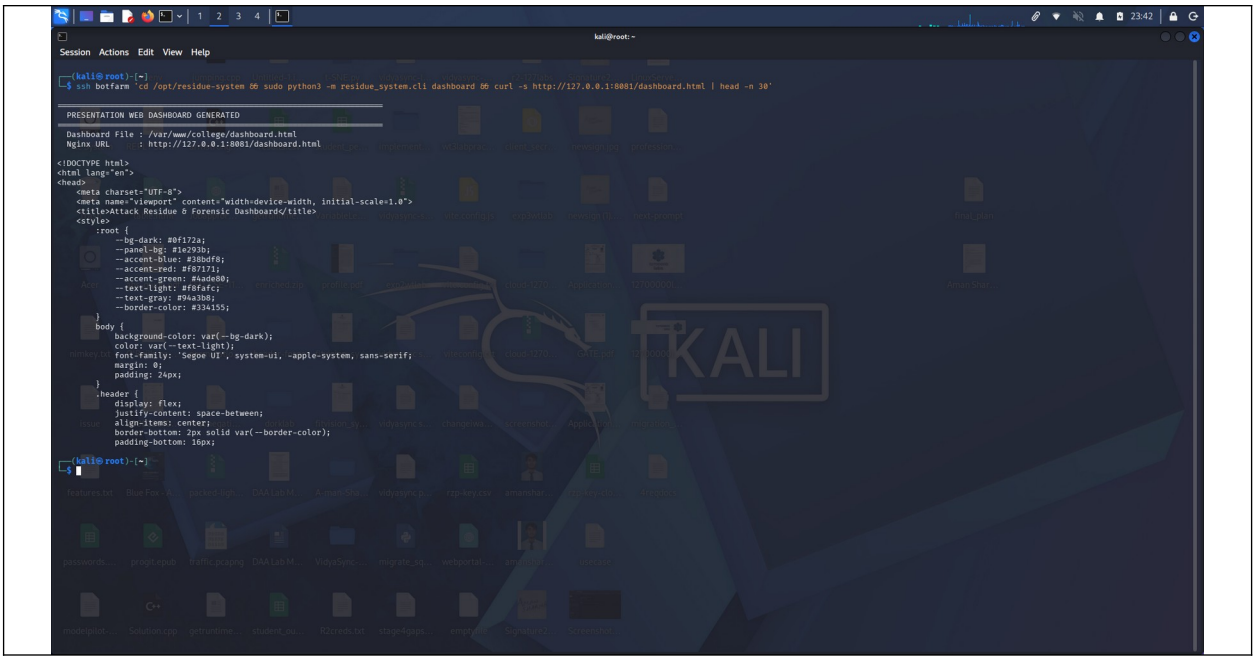
4.1 User Interfaces (UI)

The system provides two complementary user interfaces:

1. CLI Interface: Standardized colorized terminal console displaying scan progress, SHA256 diffs, severity levels (1-10), and containment logs.

2. Live Web Dashboard: Dark-mode glassmorphic HTML/JS interface at `http://127.0.0.1:8081/dashboard.html` showing real-time incident state (CLEAN vs INCIDENT), system telemetry, color-coded findings table (Red for High/Critical, Orange for Medium, Grey for Info), and live audit feeds.

Figure 4.1: Live Presentation Web Dashboard (dark-mode UI showing real-time incident telemetry, severity badges, and audit log feed).



4.2 Hardware Interfaces

Standard host keyboard, mouse, and 1080p display. Network connectivity between Kali workstation and Oracle Cloud target VM over SSH/HTTP.

4.3 Software Interfaces

Python 3.14 Standard Library (hashlib, json, os, sys, subprocess, time, uuid, http.server), Auditd 3.x Linux kernel audit daemon, Nginx 1.26 web server, Systemd init system.

4.4 Communication Interfaces

HTTP/1.1 over TCP Port 8081 for the live web presentation dashboard; SSH over Port 22 for target administration.

5. Non-Functional Requirements (NFRs)

Attribute	Target Metric / Standard	Verification & Testing Method
5.1 Performance	Detects real attack and computes state diff within < 0.45 seconds of execution.	Timed programmatically during positive compromise verification test.
5.2 Security	Isolated lab VM execution; zero real user credentials or live malware stored.	Manual code review of test scripts and artifact permission verification (0600).
5.3 Reliability	Detector runs continuously without crashing during full presentation demo.	Full end-to-end dry run execution without exceptions or memory leaks.
5.4 Accuracy	100% detection rate on planted attack artifacts and 0 false positives on negative control.	Verified via positive compromise test and negative control admin test.

6. System Design and Analysis Models

System Analysis & Data Flow Diagram (DFD Level 1):

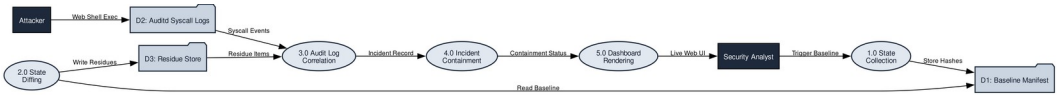


Diagram Type	Objective & Scope	Deliverable Format
Use Case Diagram	Maps Analyst, Attacker, and Target System to baseline, detection, correlation, containment, and dashboard use cases.	Graphviz high-resolution UML diagram embedded in UseCases.docx.
Data Flow Diagram (DFD)	Level 0 Context Diagram and Level 1 Process Flow (State Collection, Diffing, Correlation, Containment, UI).	Graphviz DFD Level 1 diagram embedded in SRS Section 6.
Entity-Relationship / State Model	Entity relationship schema for Baseline Manifests, Residue Objects, Auditd Evidences, and Incident Records.	Python dataclass statemodel schema (residue_system/core/statemodel.py).

7. Academic Review & Sign-Off

Project Guide / Consultant: Sarath G (Cybersecurity Trainer & Consultant) Signature: _____ Date: 30-09-2026	**Student**
Signature: _____ Date: _____	Signature: _____ Date: _____